

PrimeVector: High-Concurrency Structural Vectorization Engine

Go 1.26.2 License Research-Draft

PrimeVector is a research-based implementation of a Locality-Sensitive Hashing (LSH) engine designed for high-concurrency risk intelligence pipelines. Unlike cryptographic hashes that prioritize the Avalanche Effect, PrimeVector extracts stateful structural vectors from byte streams, preserving similarity relationships for clustering and anomaly detection.

System Architecture

The engine is built on a high-performance pipeline that transforms raw byte streams into d-dimensional structural vectors, which are then dispatched via Kafka to downstream Scala analytics services.

```
graph TD
    A[Raw Byte Stream] --> B[InputPadder]
    B --> C[ByteNgramEncoder]
    C --> D[LSHProjector]
    D --> E[VectorNormalizer]
    E --> F[KafkaDispatcher]
    F -->|Protobuf| G[Kafka Topic: risk.vectors.ingest]
    G --> H[Scala Risk Intelligence Pipeline]
    H --> I[Graph Database / Clustering]
```

Core Components

- InputPadder:** Ensures data meets minimum length requirements via cyclic repetition to prevent silent sparse vectors.
- ByteNgramEncoder:** Performs positional byte n-gram extraction (bigrams), ensuring byte ordering contributes to the feature vector.
- LSHProjector:** Implements seeded random projection onto a Gaussian hyperplane family, satisfying the formal (r_1, r_2, p_1, p_2) -sensitivity guarantee.
- KafkaDispatcher:** Handles Protobuf serialization and high-throughput dispatch to downstream consumers.

Getting Started

Prerequisites

- Go 1.26.2 or later
- Apache Kafka (for dispatcher integration)

Installation

```
git clone https://github.com/pd241008/primevector.git
cd primevector/backend
go mod download
```

Running the Backend

The backend initializes an HTTP server for health monitoring and ingestion triggers.

```
go run cmd/backend/main.go
```

Mathematical Foundation

PrimeVector targets a family of hash functions where collision probability is monotonically related to structural similarity:

$$d(x,y) \leq r_1 \implies P[h(x)=h(y)] \geq p_1 \quad d(x,y) \geq r_2 \implies P[h(x)=h(y)] \leq p_2$$

The engine utilizes **Cosine Similarity** to measure angular relationships between vectors: $\text{similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{|A| \cdot |B|}$

Integration Contract (Protobuf)

Go-to-Scala communication is governed by a strictly typed message envelope defined in Protobuf v3:

```
message VectorPayload {
  string   node_id   = 1; // Unique UUID v4
  repeated double vector = 2; // d-dimensional structural vector
  int64    timestamp = 3; // Unix epoch ms
  string   source_tag = 4; // Kafka partition key
  int32    input_len  = 5; // Original payload length
  string   engine_ver = 6; // PrimeVector build version
}
```

Project Structure

```
.
├── backend/
│   ├── cmd/backend/    # Entry point
│   ├── internal/       # Core engine logic (LSH, Encoder, Padder)
│   ├── pkg/models/     # Shared data models
│   └── go.mod           # Go module definition
├── docs/               # Research papers and specifications
└── README.md           # Project overview
```

▣ Author

Prathmesh Prakash Desai

Version 1.0 – Research Draft

▣ Project Roadmap

- ☒ **Phase 1:** Structural Vectorization Concepts & Docs.
- ☒ **Phase 2:** Backend Engine Development.
- ☐ **Phase 3:** Frontend UI Implementation.
- ☐ **Phase 4:** End-to-End Integration.